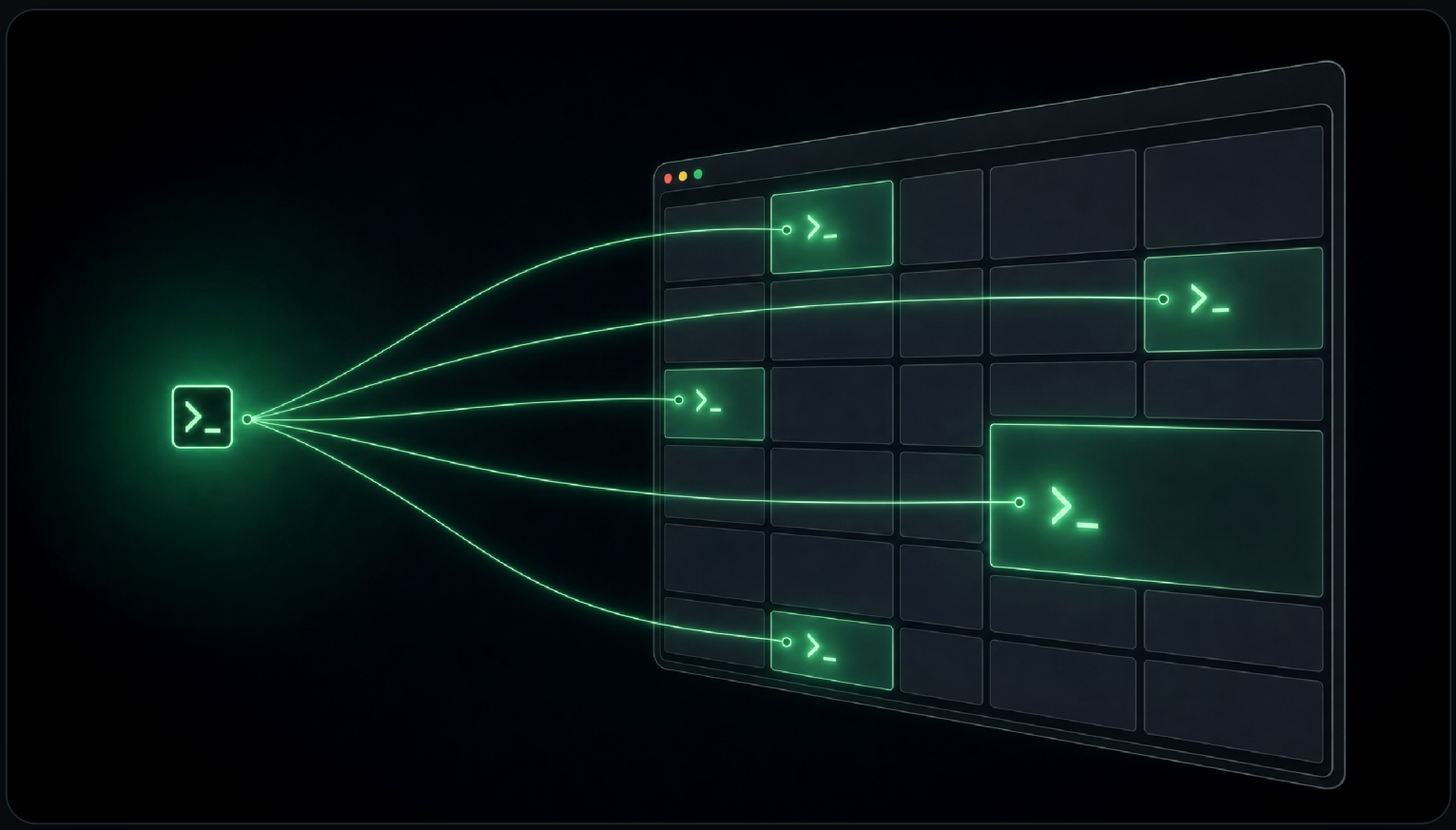
STUDYING CMUX · CAPABILITY MAP

Drive a whole fleet of agents from one terminal.

- Every terminal **addressable**
- Every agent **scriptable**
- **One orchestrator** drives them all, through cmux's CLI



CONTENTS

00 **Problem & Solution**
Three orchestration problems, three cmux solutions.

01 **The Mental Model**
Window → Workspace → Pane → Surface → Panel.

02 **The Control Loop**
send · send-key · read-screen · close-surface.

03 **Read, Decide & Get Notified**
Push the events, poll the content.

04 **Climb the Tiers**
All 31 prompts, grouped. Copy any one.

ORCHESTRATOR

1 primary agent

LIBRARY

31 ready prompts

CLIMB

5 tiers + customization

SURFACE

100% terminal-native

00 Problem & Solution

Problems come first, tools come second. Running **3-tier agent teams** (an orchestrator, team leads, and specialized workers, one team per feature) surfaced three problems. Each row reads left to right: the **problem**, then the cmux **solution**.

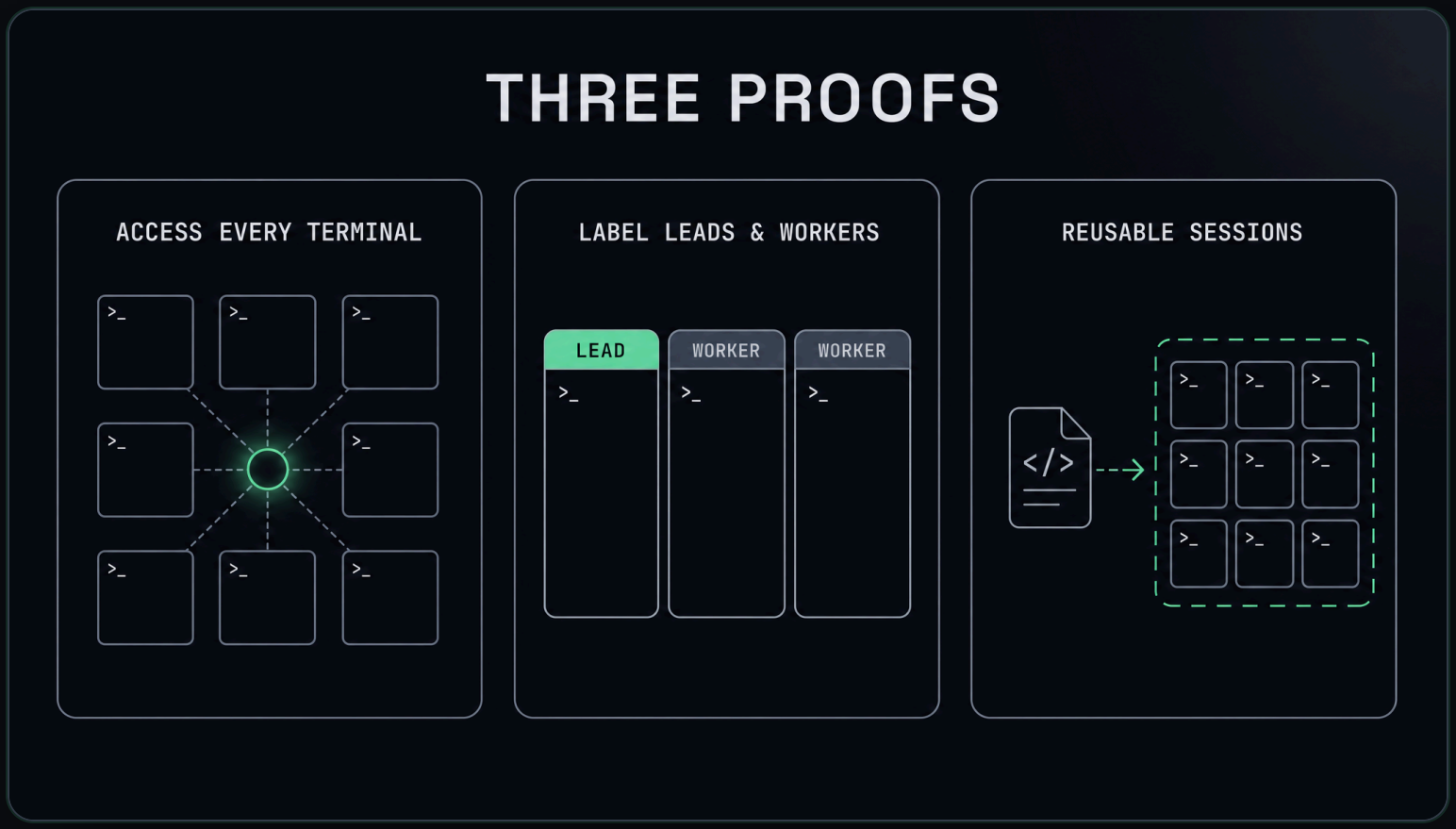


Fig. 1 · The three multi-agent orchestration problems cmux has to solve: drive every surface, see every role, boot a team from a file.

1

PROBLEM

No programmatic access means y the bottleneck

If a tool can't be driven by a CLI or API, yo nts can't operate it. A human stays in the loop, so you never reach agentic speed, le operate agents with agents.

CMUX SOLUTION

Agentic access to every terminal

Every surface is a real, addressable termi `rface:3`) exposed over a CLI and socket. One orchestrator, itself an agent, drives th e window: spawn, prompt, read, hand off. Agents operating agents.

Tiers 1-5

2

PROBLEM

An agent you can’t see is an agen can’t improve

A 3-tier team in identical gray panes is a b x. You can't tell a lead from a worker or watch the hand-offs, so you can't debug c ove the system. That's vibe coding, not engineering.

CMUX SOLUTION

Customizable panes, per role

Per-workspace color, role pill, and icon, pl pane identity: tab name, colored banner, colored prompt. One window per team, lea ne left, workers on the right. Read the whole hierarchy at a glance.

prompt 26 prompt 31

3

PROBLEM

Booting a team by hand kills agen eed

Standing up an orchestrator, leads, and w… by hand takes minutes of clicking and re-typing, and a restart wipes it. You can't sp… team per feature at the speed of agents.

CMUX SOLUTION

Reusable session files

A declarative file boots the whole roster fr… prompt (`cmux.json` `commands[]`), and crash-proof resume rebuilds it after a rest… esh team per feature in seconds.

`prompt 09` `prompt 24`

THE THESIS

Every agent is a real terminal surface you can see, and every surface is addressable (`surface:3`) and scriptable. Orchestration collapses to a handful of verbs one agent already understands.

01 The Mental Model

Everything in cmux nests in one hierarchy. Learn the five boxes and the verbs fall out of them.

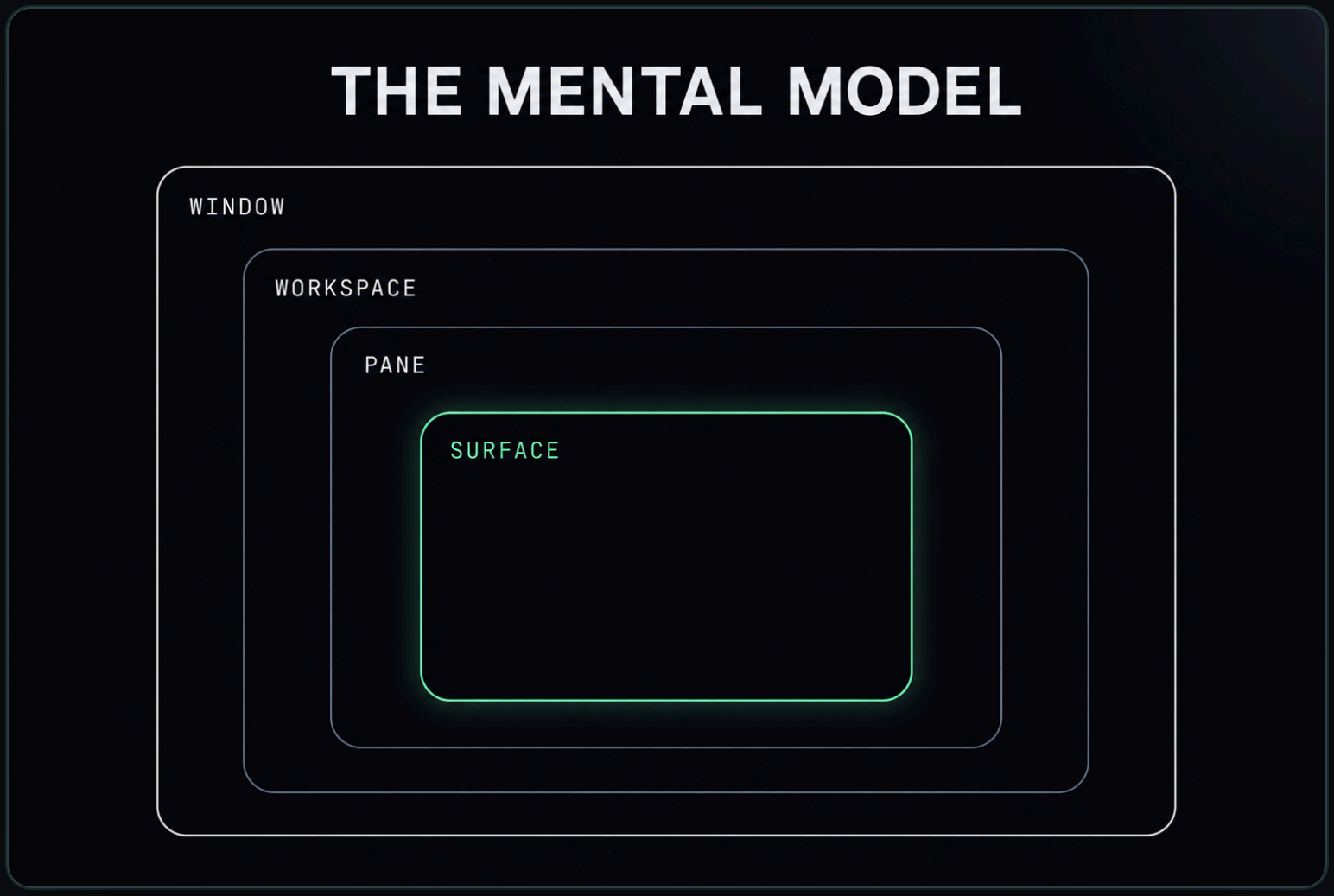


Fig. 2 · Five nested boxes: a Window holds Workspaces, which hold Panes, which hold Surfaces, each wrapping one Panel.

BOX	WHAT IT IS	THE VERB
Window	A macOS window with its own sidebar.	cmux new-window
Workspace	A sidebar entry (a “tab”).	cmux workspace create
Pane	A split region within a workspace.	cmux new-split <dir>
Surface	A tab within a pane (terminal or browser).	cmux new-surface
Panel	The content itself, created automatically.	n/a

MEMORIZE THIS

Window → Workspace → Pane → Surface → Panel. Refs like `workspace:2` and `surface:3` are handles you capture at creation and thread through every later call.

02 The Control Loop

Four verbs *are* the orchestration loop. Wrap them in a workspace and one agent is operating four others:

- **send** types text into a surface.
- **send-key** submits it (press Enter).
- **read-screen** reads what it printed: the agent's eyes.
- **close-surface** tears it down.

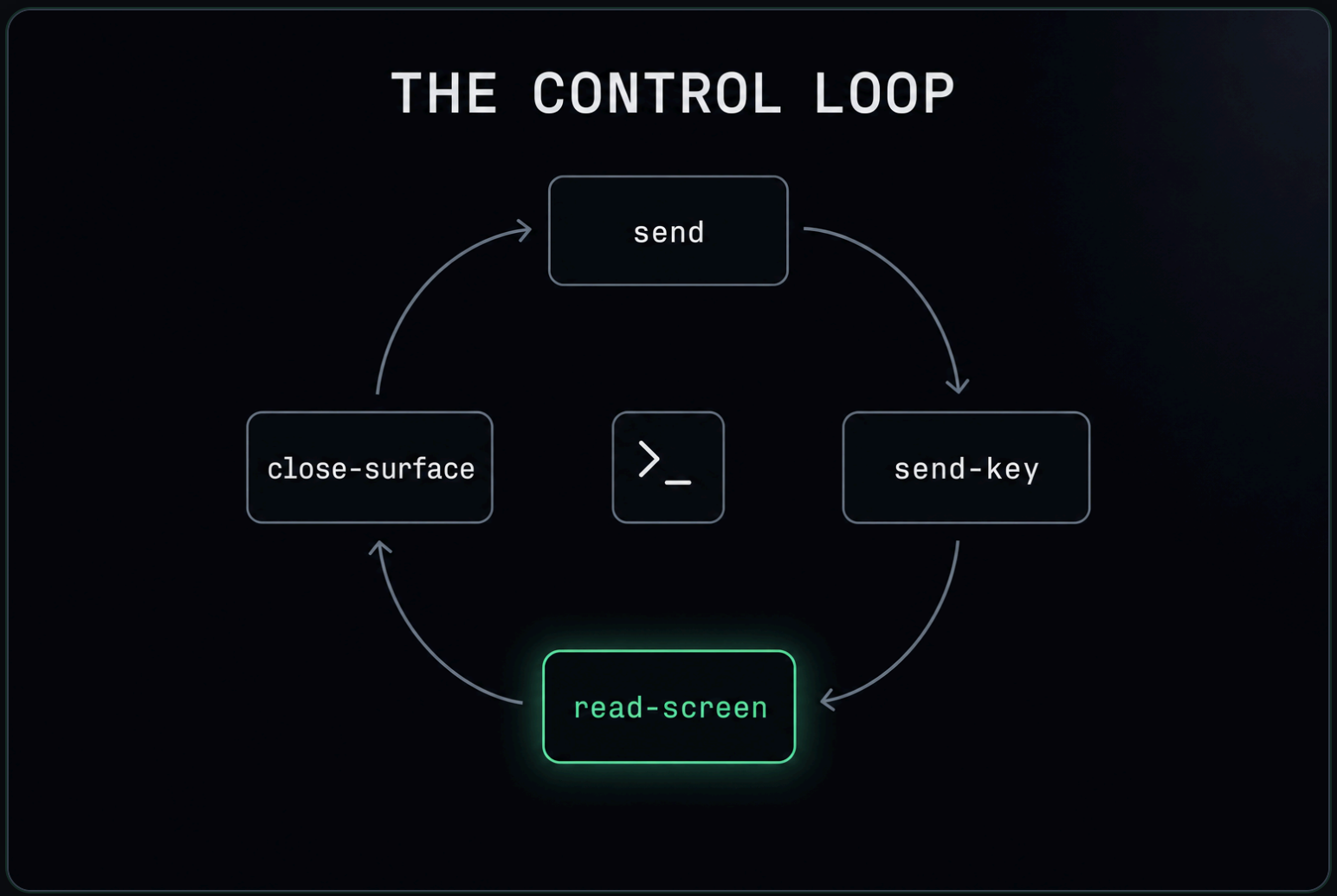


Fig. 3 · `send` / `send-key` push into each surface, `read-screen` is the agent's eyes, `close-surface` ends it.

THE ENTIRE CONTROL LOOP · FOUR VERBS

```
cmux send        --surface surface:3 "claude"     # type into a terminal
cmux send-key    --surface surface:3 enter        # press a key (submit)
cmux read-screen --surface surface:3              # read what it printed (its eyes)
cmux close-surface --surface surface:3            # shut it down (teardown)
```

ONE GOTCHA

`send` types; `send-key` submits. There is no modifier chord. To stop a running agent, `close-surface` it cleanly.

03 Read, Decide & Get Notified

An orchestrator does two things a plain script can't:

- **Reads & decides**: what did the agent actually say? *Poll* it with `read-screen`.
- **Gets notified**: the instant something happens. cmux *pushes* discrete events via `events` / `wait-for`.

The rule: **push** the events, **poll** only the free-form content you must inspect.



Fig. 4 · `cmux events` / `wait-for` are the doorbell (push); `read-screen` is how you read the room (poll).

TO KNOW…	USE	PUSH OR POLL
an agent finished / needs input	cmux events --name agent.hook	**push**
block until task X signals done	cmux wait-for X	**push** (blocks)
any notification, durably	cmux events --category notification --reconnect	**push**
what the agent actually wrote	cmux read-screen --surface <ref> --scrollback	poll

PRIVACY-RESPECTING DOORBELL

The event payload carries `workspace_id` / `surface_id` / `notification_id` and content lengths, but the title and body are redacted. Fetch the text by reading the surface.

04 Climb the Tiers

The capability library: 31 natural-language prompts that climb from “open one workspace” to “run an autonomous, browser-verified software factory”, then customize the whole thing. Hit **Copy prompt** and hand any one to your orchestrator.

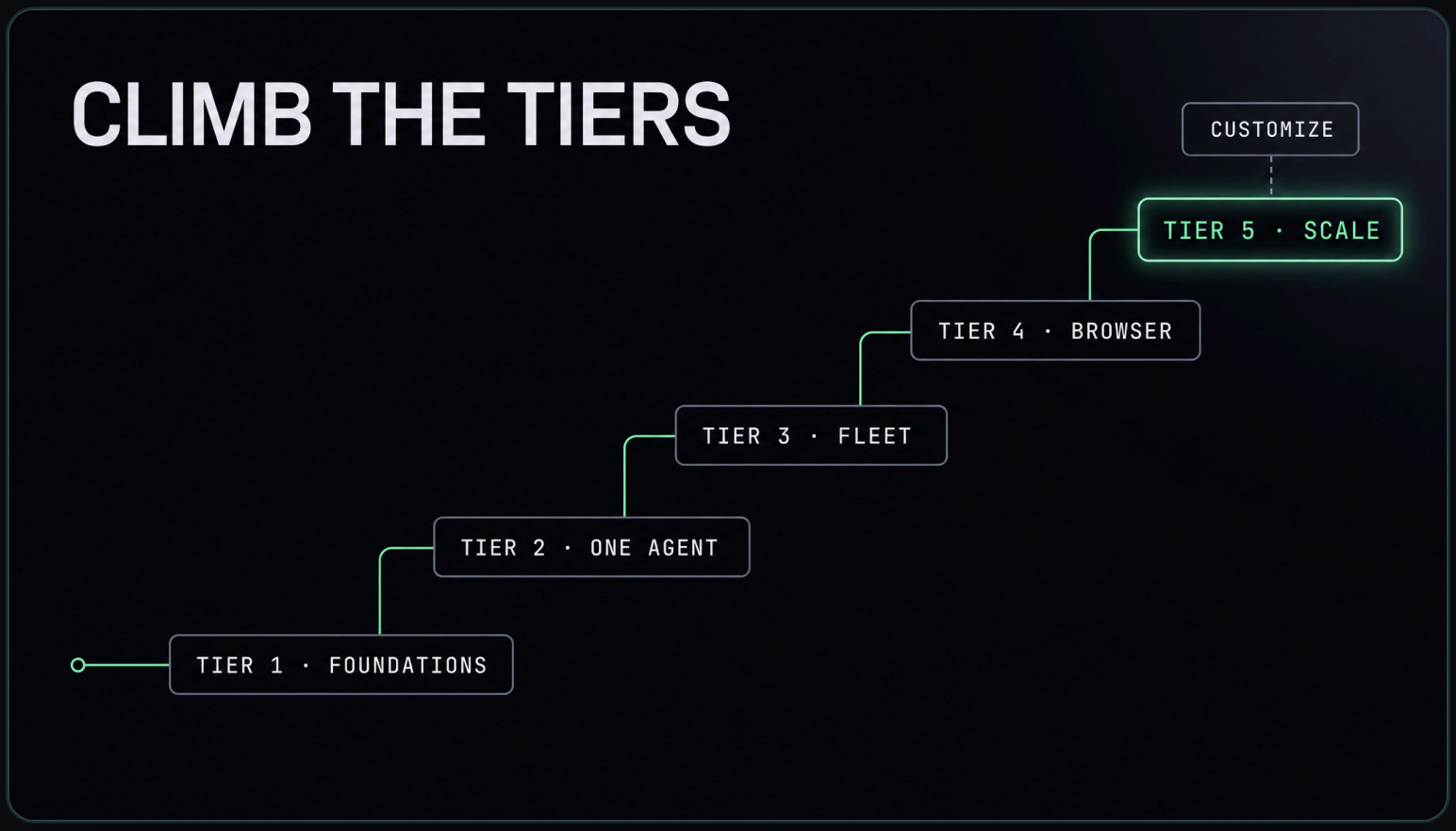


Fig. 5 · Five tiers of orchestration plus a customization track. Each prompt is one rung you can copy and run.

Tier 1 · Foundations

The atomic loop: create a surface, type into it, read it back, tear it down. No agents yet, just the orchestrator and terminals.

01 Hello, Workspace COPY PROMPT

Open a workspace, run a command, read the result back.

› In cmux, open a new workspace called **scratch** in a throwaway temp directory, run `uname -a` in it, then tell me exactly what it printed.

02 Read It Back

COPY PROMPT

Pull output that scrolled off-screen out of scrollback.

› Open a workspace in a fresh temp directory, create 60 throwaway files in it, list them with `ls -la`, then read the output back from scrollback and tell me how many files there are — and name three.

03 Split Into a Grid

COPY PROMPT

Turn one workspace into a live 2×2 of terminals.

› Build me a monitoring dashboard in one cmux workspace called **dash** in a temp directory: split it into a 2×2 grid of terminals. Top-left run `htop`, top-right `watch -n1 date`, bottom-left tail a temp log file it owns, and leave the bottom-right as a plain shell.

04 Map the World

COPY PROMPT

Dump the full window / workspace / pane / surface tree + per-surface CPU/mem.

› Show me everything cmux is running right now — every window, workspace, pane and surface — and for each terminal tell me the process, CPU, and memory. Give it to me as a tidy report, and flag anything using more than 50% CPU.

05 Tidy Up

COPY PROMPT

Rename, flash for attention, close panes, then the workspace.

› Take the **dash** workspace from before: rename it to **retired**, flash it so I can spot it, then close its extra panes one by one, and finally close the whole workspace.

Tier 2 · Driving One Agent

The pivot: the terminal an agent drives can itself contain another agent. This is the whole idea, in miniature.

---

06 Launch One Agent

COPY PROMPT

Boot Claude Code in a pane and prompt it.

> › Open a fresh pane, start **Claude Code** in it, ask it to write a haiku about terminals, give it a few seconds, then read me back exactly what it wrote.

07 Keys & Credentials

COPY PROMPT

Inject .env so an agent comes up authenticated, without leaking secrets.

> › Spin up a workspace that has my API keys from `.env` loaded into its environment, launch the **pi** coding agent there, and ask it to confirm it can reach a model. Read me what it says — but never print my actual keys.

08 Two Agents, One Question

COPY PROMPT

Claude Code + pi side by side, same prompt, compared.

> › Put **Claude Code** and **pi** side by side in one workspace. Ask both the same question — *'In two sentences, what's the difference between a process and a thread?'* — then read both answers and tell me where they agree and where they differ.

09 Declarative Boot

COPY PROMPT

A reusable agent layout as code (`cmux.json` or `--layout`).

> › Add a reusable **AI Pair** layout to my cmux config: a left pane running Claude Code and a right pane that's a plain shell in the current project. Validate it, then launch it.

10 Native Sessions & Resume

COPY PROMPT

Spawn a provider-aware agent surface that resumes after restart.

> Spawn Claude Code as a **native cmux agent session** — not just the word 'claude' typed into a shell — and set things up so that if cmux restarts, this session resumes exactly where it left off.

Tier 3 · Orchestrating a Fleet

One orchestrator, many agents. Broadcast, shard, watch, react, and tear down at scale.

---

11 The 2×2 Fleet ★

COPY PROMPT

Four agents (Claude, Codex, Gemini, pi), one broadcast task, read all four.

> Stand up four coding agents in a 2×2 grid — **Claude Code**, **Codex**, **Gemini**, and **pi** — in a workspace called **fleet** with my `.env` loaded. Give all four the same task: *'List the top 3 security risks you can find in this repo.'* Then read all four answers back and tell me which risks they agree on.

12 Fan-Out / Fan-In

COPY PROMPT

Shard a job across agents, then merge their outputs.

> Split this repo's work four ways. Give each of four agents one top-level directory — `src/`, `tests/`, `docs/`, and `scripts/` — and have each summarize *only its directory* in five bullets. Then collect all four summaries into a single map of the codebase.

13 Live Status Board

COPY PROMPT

Colored status pills + progress bars + logs, one per agent.

> Run four agents on four refactors, but give each its **own sidebar workspace**, and keep the sidebar live: show each one's status as a colored pill (**working** = blue, **done** = green, **error** = red), a progress bar, and a one-line log of what it's doing. I want to glance at the sidebar and know the whole fleet's state.

14 Reactive Loop

COPY PROMPT

Drive off cmux's event stream instead of polling screens.

> Here's a queue of ten tasks and three agents. Don't poll the screens. **Subscribe to cmux's event stream**, and the moment an agent finishes (its surface notifies or goes idle), read that agent, hand it the next task from the queue, and keep going until the queue is empty.

15 Race & Notify

COPY PROMPT

Race three agents; the first to finish pings you, the rest are stopped.

> Give the same hard task to three agents as a race. Wire each so it fires a cmux notification the moment it's done. Jump me to whichever finishes first, take its answer, and stop the other two.

16 Scale & Teardown

COPY PROMPT

Rank by CPU, prune stragglers, reassign their work, reclaim the panes.

> Launch six agents on this batch of tasks. After two minutes, look at which ones are still chewing CPU with nothing to show, close the slowest two, reassign their unfinished tasks to agents that already finished, and when the batch is done, tear the whole fleet down.

Tier 4 · Terminals Meet the Browser

A cmux surface can be a browser, with a Playwright-style API. Now agents build, run, and verify in one window.

---

17 Agent + Live Preview

COPY PROMPT

A coding agent on the left, a live preview of its app on the right.

> In one workspace, put **Claude Code** on the left building a tiny web app, and an embedded **browser** on the right pointed at its dev server (`http://localhost:5173`). Each time the agent changes the code, reload the preview so I can watch the page evolve.

18 Agent Drives the Browser

COPY PROMPT

Snapshot the DOM, click, fill, screenshot to prove it worked.

> Open `https://example.com` in a cmux browser, snapshot the page so you know what's there, click the **'More information...'** link, wait for the new page, and screenshot the result so I can confirm it worked.

19 Self-Verifying Agent

COPY PROMPT

Read the browser console / errors, fix, re-check until clean.

> Agent: build a small page and run its dev server. Then drive the cmux browser against your own UI — load the page, read the browser **console and error log**, and if anything's broken, fix the code and re-check. Loop until the console is clean, then screenshot the working page.

20 Capture & Replay Auth

COPY PROMPT

Log in once, save the session, replay it across every agent run.

> I'll log into the app once in a cmux browser. Save that whole browser session to a file. Then, for every future agent run, load that saved state first so the agent's browser is already authenticated — no logging in again.

Tier 5 · Scale, Remote & Resilience

The largest moves: multiple windows, organized groups, remote and cloud agents, restart survival, and the capstone.

---

21 Multi-Window Command Center

COPY PROMPT

Spread fleets across macOS windows, route work per window.

> Don't cram everything into one window. Open three macOS windows — **frontend**, **backend**, and **ops** — put a small agent fleet in each, route the right tasks to each window, then gather all the results into one summary for me.

22 Organized Sidebar at Scale

COPY PROMPT

Twenty agents kept legible in named, colored, collapsible groups.

> I'm about to run twenty agents. Keep my sidebar legible: organize them into named, colored, collapsible groups — **reviewers** (blue), **fixers** (green), **testers** (amber) — so I can collapse a group and scan the rest.

23 Remote & Cloud Fleets

COPY PROMPT

Agents on SSH hosts and cloud VMs, driven by the same verbs as local.

> Run agents where the code actually lives. Open an **SSH workspace** on my dev box and spin up a fresh **cloud VM**, launch a coding agent in each, and drive them with the exact same `send` / `read-screen` commands you use locally.

24 Crash-Proof: Resume the Fleet

COPY PROMPT

Quit and relaunch; the layout rebuilds and agents resume their sessions.

> Make my fleet restart-proof. Install the resume hooks for every agent, start several agents working, then quit and relaunch cmux — and confirm the layout came back **and** each agent picked up its own session where it left off.

25 The Software Factory ★

COPY PROMPT

Plan → fan out → test → browser-verify → notify → collect diffs → teardown.

> Be my build lead. Take this feature request and ship it: **plan** the work, open a **window per subsystem**, **fan out** the implementation to a fleet of agents (roughly one per file), keep a **live status board** so I can watch, **run the tests** as each agent finishes, **browser-verify** the running app, **notify** me with the result, **collect every agent's diff** into one PR-ready bundle, and **tear the fleet down** when it's green.

Customization

Per-workspace identity is scriptable on the fly, the terminal's look is a live-reloaded global, and the app's actions, layouts, and sidebar are yours to redefine, globally or per-repo.

---

26 Color-Code & Label the Fleet

COPY PROMPT

Per-workspace color, role pill, icon, and name, set live from a script.

> I'm about to run a dozen agents. Make them scannable: give each workspace its own **color**, a **role pill** with an **icon** (reviewer / fixer / tester), and a clear name, so I can tell them apart at a glance.

27 Theme & Feel

COPY PROMPT

Font, theme, opacity, cursor via Ghostty config; live-reload, no restart.

> Change the terminal's whole look: switch to a darker theme, bump the font a touch, dial the background to ~92% opacity, and apply it **without restarting**.

28 Custom Actions & Buttons

COPY PROMPT

Your own verbs in the Command Palette, tab bar, and **+** button.

> Give me one-tap ways to spawn agents: a **Codex** entry in the Command Palette, a **Codex** button on the tab bar, and make the **+** button's right-click menu offer 'New Terminal / New Browser / Spawn Codex'.

29 Reusable Layouts & Dock Controls

COPY PROMPT

Named `commands` layouts + a right-sidebar Dock of tools.

> Two things: a one-command **dev** workspace (dev server on the left, live browser preview on the right), and a **Dock** in the right sidebar with quick buttons for lazygit, a test watcher, and the cmux Feed.

30 Make It Yours

COPY PROMPT

Per-repo `.cmux/cmux.json` + a hot-reloading custom sidebar bound to live data.

> Two things. Make this repo's cmux behavior travel with it (its own layouts/actions/dock). And build me a **custom sidebar** that lists my workspaces with a live clock and each one's PR status, where clicking a row jumps to it.

31 Identify Individual Panes

COPY PROMPT

Per-pane identity: tab name + colored ANSI banner + colored prompt.

> Split a workspace into 4 panes and make each one instantly identifiable — give each its own tab name, a bold colored banner at the top, and a matching colored prompt, so I can tell **reviewer / fixer / tester / builder** apart at a glance.

One primary agent. A whole window of terminals. Full programmatic control.